1501116 & 15011001 Computer Programming

Chapter 9: C# Array

Academic Year 2026/01

School of Applied Digital Technology, Mae Fah Luang University

Revised version by: Patcharaporn Panwong
Paweena Suebsombut
Sujitra Arwatchananukul

Original version by: Surapong Uttama

Lesson Plan

Week	Weekly Summary
Week 10 (5-9 Oct 20206)	**Chapter 7:** Iteration Statement: Loop Part 2
Week 11 (12-16 Oct 2026)	**Chapter 8:** Methods
Week 12 (19-23 Oct 2026)	**Chapter 9:** Array
Week 13 (26-30 Oct 2026)	**Chapter 10:** Class and Object
Week 14 (2-6 Nov 2026)	Group Project Implementation
Week 15 (9-13 Nov 2026)	**Group Project Presentation** ★★★
Week 16 (16-20 Nov 2026)	**Group Project Presentation** ★★★
Final Exam (Thursday 26 November 2026, 09:00 - 12:00)	

FIRST SEMESTER 2026

The Online Teaching Evaluation System

NOW OPEN . . .

You can complete the online teaching evaluation for all subjects from now until November 22, 2026

Reminders

The students who do not complete the online teaching evaluation before November 8, 2026.
"The REG system will be locked."

Scan to Evaluate

Contact Info

Division of Quality Assurance and Curriculum Development.
AS Building, 4th Floor
E-mail : qa.eval@mfu.ac.th
Tel. 053 916 407



Link:
https://evaloutcome.mfu.ac.th/login

Outline

- Review: Methods
- Arrays overview (single-dimensional arrays)
- Foreach loop in Array
- Two-dimensional arrays

Review

Method

a group of statements for a specific purpose

1. What is Method?

Examples of its advantage:
1) Easier to read and understand
2) Code reuse

2. Types of Method

- **By Source**
  1. Built-in / Standard C# Methods / Predefined method
  2. User-defined Methods

- **By Parameter and Return Data**
  1. **No** parameter + **No** return data
  2. **No** parameter + **Return** data
  3. Parameter + **No** return data
  4. Parameter + **Return** data

Calling / Invoking Method

Type: No parameter + No return data

```
using System;
public class Program
{
    public static void Main()
    {
        SayHi() ;
    }
    public static void SayHi()
    {
        Console.WriteLine("Hi!!!");
    }
}
```

Main method

SayHi method

call

Output / Result

```
Hi!!!
```

Type: Parameter + No return data

```
using System;
public class Program
{
    public static void Main()
    {
        SayHi("Teddy") ;
    }
    public static void SayHi(string name)
    {
        Console.WriteLine("Hello, " + name);
    }
}
```

call

Output / Result

```
Hello,  Teddy
```

Calling / Invoking Method

Type: No parameter + Return data

```
using System;
public class ITschool{
    public static void Main(string[] args){
        Console.WriteLine("IT school has {0} majors.",
GetNumMajor() );
    }
    public static int GetNumMajor() {
        int numMajor = 4;
        return numMajor ;
    }
}
```

call

return

Output / Result

```
IT school has 4 majors.
```

Type: Parameter + Return data

```
using System;
public class Program{
    public static void Main(string[] args){
        Console.Write("Enter the first number: ");
        int tmpNum1 = Convert.ToInt32(Console.ReadLine());
        Console.Write("Enter the second number: ");
        int tmpNum2 = int.Parse(Console.ReadLine());

        Console.WriteLine("Sum = " + Sum(tmpNum1, tmpNum2) );
    }

    public static int Sum(int num1, int num2) {

        return num1 + num2 ;
    }
}
```

call

return

Output / Result

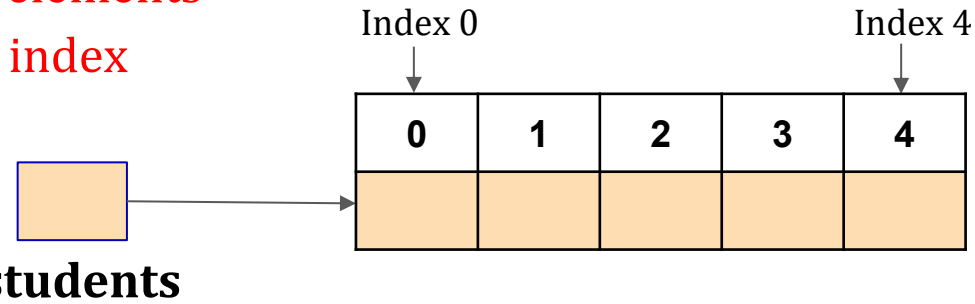
```
Enter the first number: 9
Enter the second number: 4
Sum = 13
```

Arrays overview

(Single-dimensional arrays)

Array

- C# array is an object of base type **System.Array**.
- The **[ ]** characters follow the data type to denote that the variable is an array
- An array is a collection of values of the **same data type**
- Individual variables are called elements
- Elements accessed through an index
- Index starts at 0



** The following figure shows the **array** named as **students**.

** The **students array** contains **5 elements**.

Array Declaration and Creation

- **Declaration:**

```
Data Type [] variable_name;
```

```
string[] students; //can store string values
int[] nums; //can store int values
float[] values; //can store float values
```

- **Creation size of array:**

```
variable_name = new type[size];
```

```
students = new string[3];
nums = new int[5];
values = new float[10];
```

Array Declaration and Creation

- **Combine declaration & creation command**

```
string[] students = new string[3];
```

*** Then, **students array** contains **3 elements**.*

students[0], students[1], students[2]

students

0	1	2

Initialization of Array

- Initialization = Defining the starting values to an array


  - int[] num = new int[3] {1,2,3}; *//create array size 3 elements and set initial value*


  - int[] num = {1,2,3}; *//create and initialize*


  - int num[2] = {1,2}; *//wrong, size is not needed*

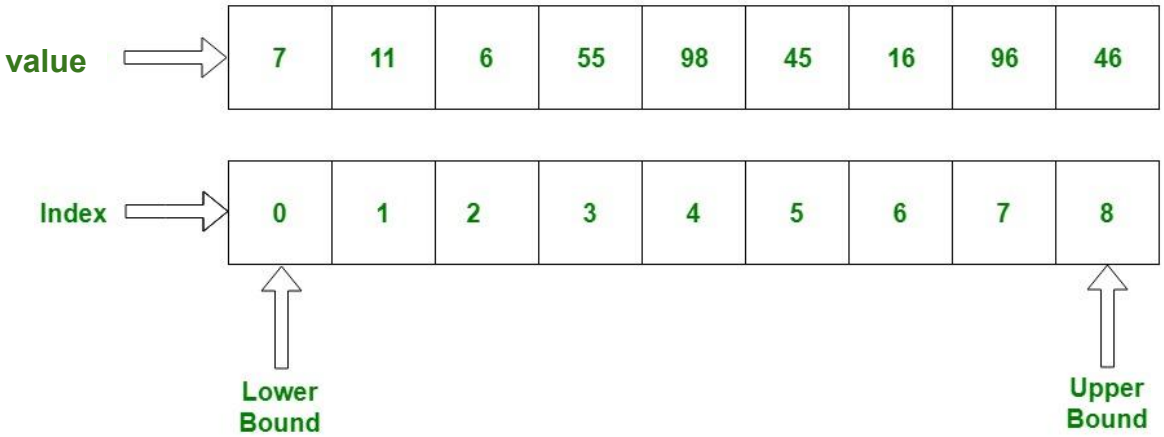


- Is initialization necessary?
  - Require initialization → when you know exactly the values of array's element

Example: Initialization of Array

```
// defining array with 9 elements which indicates the size of an array
// int[] value  = {7, 11, 6, 55, 98, 45, 16, 96, 46};
```

```
int[] value  = new int[9] {7, 11, 6, 55, 98, 45, 16, 96, 46};
```



C# | Arrays - GeeksforGeeks

Processing Array

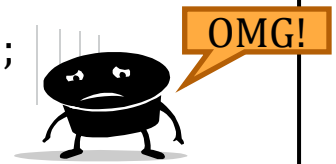
- Loop could help us dealing with array.

```
int[] a = {77,12,54};
int sum = 0;
//loop for summing each element in array
//a.Length => number of elements in array
for(int i=0; i<a.Length; i++)
{
        sum = sum + a[i];
}
```



```
int[] a = {77,12,54};
int sum = 0;
//calculate summation of each element in array

sum = a[0] + a[1] + a[2];
```





Easier???
Imagine we have 1000 elements???

Array as Parameter of Method

- **Two ways** for sending an array to a method
    1. Sending **each member** of an array
    2. Sending a **whole array**



Ex. Array as Parameter of Method

```
using System;
public class test{
      public static void Main(string[] args) {
            int[] arr = { 1, 2, 3 };
            // call addition method
            Console.WriteLine("value1 + value3 = {0}" , Add2Values( arr[0], arr[2] ));
            // call summation method
            Console.WriteLine("Summation of all values is {0}" , Summation( arr ));
      }
      // define a method of add 2 values
      public static int Add2Values( int value1, int value2 ) {
            return ( value1 + value2 );
      }

      // define a method summation to add a holding array value
      public static int Summation( int[] a ) {
            int sum = 0;
            for (int i = 0; i < a.Length; i++) {
                  sum += a[i];
            }
            return  sum;
      }
}
```

Example: Sending member(s) of an array

Result:
value1 + value3 = 4

Return the value of summation to the main method

Send -> value arr[0]=1 to variable value1
value arr[2]=3 to variable value2

Ex. Array as Parameter of Method

```
using System;
public class test{
      public static void Main(string[] args) {
            int[] arr = { 1, 2, 3 };
            // call addition method
            Console.WriteLine("value1 + value3 = {0}" , Add2Values( arr[0], arr[2] ));
            // call summation method
            Console.WriteLine("Summation of all values is {0}" , Summation( arr ));
      }

      // define a method of add 2 values
      public static int Add2Values( int value1, int value2 ) {
            return ( value1 + value2 );
      }

      // define a method summation to add a holding array value
      public static int Summation( int[] a ) {
            int sum = 0;
            for (int i = 0; i < a.Length; i++) {
                  sum += a[i];
            }
            return sum;
      }
}
```

Example: Sending a whole array

Return the value of summation to the main method

Sending a holding array members

Result:

Summation of all values is 6

Return Array from Method

- **Method** can return only one value
- **C#** supports returning of multiple values by using array



Ex. Return Array from Method

```
using System;
public class test {
      public static void Main(string[] args) {
            int[] a = SetNumber();
            // show values of array a
            for (int i = 0; i < a.Length; i++) {
                  Console.WriteLine("a[" + i + "] is " + a[i]);
            }
      }

      public static int[] SetNumber(){

            int[] a = { 1, 2, 3, 4, 5 };
            return a;
      }
}
```

Return in a holding array members

Call method Setnumber()

Ex. Return Array from Method (Cont.)

```
using System;
public class test {
      public static void Main(string[] args) {
            int[] a = SetNumber();    //a = { 1, 2, 3, 4, 5 };
            // show values of array a
            for (int i = 0; i < a.Length; i++) {
                  Console.WriteLine("a[" + i + "] is " + a[i]);
            }
      }

      public static int[] SetNumber(){

            int[]  a = { 1, 2, 3, 4, 5 };
            return a;
      }
}
```

Result:

a[0] is 1
a[1] is 2
a[2] is 3
a[3] is 4
a[4] is 5

“foreach” loop in Array

Using “foreach” loop in arrays

- **Foreach** loop provides a simple, clean way to iterate through the elements of an ***collection or an array*** of items.
- **Foreach** statement processes elements in increasing index order, starting with index 0 and ending with index Length - 1
- **Foreach** cannot be used to change the contents of the array

Syntax of foreach Loop:

```
foreach (Data_Type variable_name in Collection_or_array_Object_name)
{
   //body of foreach loop
}
```

C# | Using foreach loop in arrays - GeeksforGeeks

Example: “foreach” and “for” loop

Normally, using “**foreach**” no need of using a counter

```
int[] numbers = { 4, 5, 6, 1, 2, 3, -2, -1, 0 };
foreach (int i in numbers)
{
    Console.Write("{0} ", i);
}
// Output: 4 5 6 1 2 3 -2 -1 0
```

Normally, using “**for**” with array needs a counter

```
int[] numbers = { 4, 5, 6, 1, 2, 3, -2, -1, 0 };
for(int i =0; i< numbers.Length; i++)
{
    Console.Write("{0} ", numbers[i]);
}
// Output: 4 5 6 1 2 3 -2 -1 0
```

Exercise #1

- Given an array below, change the “for” statement to **“foreach”**

```
int[] numbers = {1,2,3,4,5};
int sum=0;
for(int i=0; i<numbers.Length; i++){
        sum = sum + numbers[i];
}
```

Answer



Useful Properties and Methods

Useful Properties and Methods

Example1:

```
// Single-dimensional arrays

int [] arr = new int[3] {1,2,3};
```

- **Length:** Return the number of elements
  - Console.WriteLine(arr.Length); // The output is 3

Example2:

```
// Two-dimensional array.
int[,] array2Da = new int[4, 2] { { 1, 2 }, { 3, 4 }, { 5, 6 }, { 7, 8 } };
```

- **Rank:** Return the number of the dimensions of the array
  - Console.WriteLine(array2Da.Rank); // The output is 2
- **GetLength(0):** The method returns the size of the 1st dimension of the array
  - Console.WriteLine(array2Da.GetLength(0)); // The output is 4
- **GetLength(1):** The method returns the size of 2nd dimension of the array
  - Console.WriteLine(array2Da.GetLength(1)); // The output is 2

*** Lots of other methods in System.Array*

Array.Rank Property (System) | Microsoft Docs

Array.GetUpperBound(Int32) Method (System) | Microsoft Docs

Multidimensional Arrays

- Two Dimensional Arrays

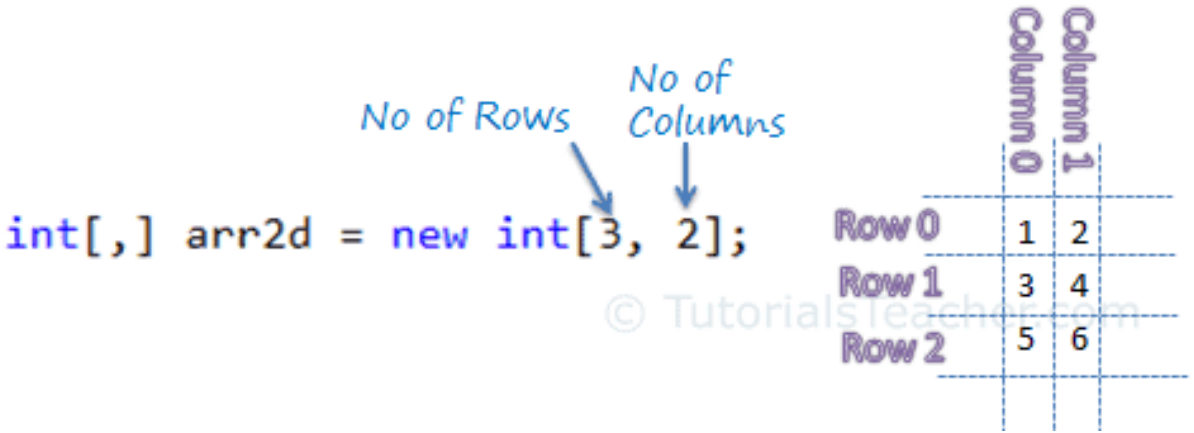
Multidimensional Arrays

- C# supports multidimensional arrays up to 32 dimensions
- The multidimensional array can be declared by adding commas in the square brackets.
  - For example
    - **[,]** declares **two-dimensional array** `// int[,] arr2d;`
    - **[, ,]** declares **three-dimensional array** `// int[, ,] arr3d;`
    - **[, , ,]** declares **four-dimensional array** `// int[, , ,] arr4d ;`
    - and so on.

C# Multidimensional Arrays: 2D, 3D & 4D (tutorialsteacher.com)

Two-dimensional Array

- Single-dimensional array → 1D → 1 index → vector
- Two-dimensional array → 2D → 2 indexes → table, matrix
- In a 2D array, the **first** index is the **row** dimension, the **second** index is the **column** dimension.



C# Multidimensional Arrays: 2D, 3D & 4D (tutorialsteacher.com)

Two-dimensional Array

- Following figure is a 2-dimensional array, which contains
  - 3 rows and 4 columns

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0,0]	a[0,1]	a[0,2]	a[0,3]
Row 1	a[1,0]	a[1,1]	a[1,2]	a[1,3]
Row 2	a[2,0]	a[2,1]	a[2,2]	a[2,3]

C# - Multidimensional Arrays (tutorialspoint.com)

Two-dimensional Array

Syntax:

```
Type[ , ] variable = new Type[row,column];
```

Example:

```
int[,] array2D = new int[4, 2];
//create a two-dimensional array of 4 rows and 2 columns in integer data type

string[,] array2Da = new string[3, 2]
//create a two-dimensional array of 3 rows and 2 columns in string data type
```

Multidimensional Arrays - C# Programming Guide | Microsoft Docs

2D array with initialization

- In the following example, you may initialize the array upon declaration.

Example:

```
// Two-dimensional array.
int[,] array2D = new int[,] { {1,2}, {3,4}, {5,6}, {7,8} };

// The same array with dimensions specified.
int[,] array2Da = new int[4, 2] { {1,2}, {3,4}, {5,6}, {7,8} };

// A similar array with string elements.
string[,] array2Db = new string[3, 2] { {"one","two"}, {"three","four"},
                                        {"five","six"} };
```

Multidimensional Arrays - C# Programming Guide | Microsoft Docs

2D array with initialization

Example:

- `int[,] a = { {11,12} , {21,22} };`

11	12
21	22

	col 0	col 1
row 0	a[0,0]	a[0,1]
row 1	a[1,0]	a[1,1]

- `int[,] a = { {11,12,13} , {21,22,23} };`

	col 0	col 1	col 2
row 0	11	12	13
row 1	21	22	23

2D array - Coding sample

```
public class MyArray {
      public static void Main(string[] args) {
         /* an array with 5 rows and 2 columns*/
         int[,] a = new int[5, 2] { {0,0}, {1,2}, {2,4}, {3,6}, {4,8} };
         int r, c;

         /* output each array element's value */
         for (r = 0; r < a.GetLength(0); r++) {
            for (c = 0; c < a.GetLength(1); c++) {
               Console.WriteLine("a[{0},{1}] = {2}", r, c, a[r,c]);
            }
         }
      }
}
```

//Nested Loop

0	0
1	2
2	4
3	6
4	8

Results

```
a[0,0] = 0
a[0,1] = 0
a[1,0] = 1
a[1,1] = 2
a[2,0] = 2
a[2,1] = 4
a[3,0] = 3
a[3,1] = 6
a[4,0] = 4
a[4,1] = 8
```

C# - Multidimensional Arrays (tutorialspoint.com)

Summary

- What is Array? Why do we need it?
- Single-dimensional arrays
- “foreach” loop in array
- Useful properties and methods
- Two-dimensional arrays

Announcement

Group Project (25%)

- Num# students **1 group = 3 - 5 students**
- Form a group with only your classmates from same lecture class
  - Example: Your lecture is on Monday (ONLY section 1)

- Present Project: **10 minutes /group**
- Question and Answer: **5 minutes /group**

- **Console application (No GUI interface)**
  - **Any topics** e.g., noodle shop, library, ticket reservation, and so on

The Group Project Evaluation Matrix (25% or 25 Points)

Computer Programming Subject: Project Rubric (25%)

if showing up on presentation day but does not present --> presentation score = 0
if missing for presentation day --> presentation score = 0
if missing and/or doing nothing for project (reported by your teammate) --> group project = 0

Evaluation Criteria									Total (25 pts)
Completeness (5 pts)	Logic & Technical Basic						Complexity (2 pts)	Presentation (3 pts)	
	if-else (2 pts)	switch case (2 pts)	Function(s) / Method(s) (4 pts)	Loop (3 pts)	Array (2 pts)	Add input data to array (2 pts)			
[1]. all complete(no error) = 5, [2]. logic error = 3, [3]. project cannot execute/run = 1	-1 when there is no statement inside	-1 when there is no statement inside	-1 when there is no calling from Main()	-1 when repeat only one round	-1 when contains only one element	-2 if no data is entered for each array element	Basic (similar to the lab) = 1 points Extended from the lab with full functions (Self-study / Wow) = 2 points	-1 if can't answer all questions -1 if begin your presentation late or go over time Remark: Case1: missing presentation = 0 points Case2: showing up on presentation day but does not present = 0 points	

To understand the grading fully, please read the Group Project Rubric here.

Final Project Presentation (25%)

- To understand the grading fully, please read the Group Project Rubric here.
  - Remark: Do not forget to read FAQ for group project & presentation!!!

Condition	Score	Outcome
Project Missing (No group and No presentation)	-25	The student does not belong to any group, does not attend and present on presentation day, and has no project submission. **The Group Project Score = 0.**
Project Missing + Non-Participation (No contribution)	-25	The student belongs to a group, but the student is reported by a teammate as having done nothing for the project. **The Group Project Score = 0.**
Presentation Absence	-3	If the student is missing for presentation day but other members confirm that you helped and worked on the project, **your Presentation Score = 0.**
The student who shows up on the presentation day but does not present.	-3	Every student in the team must present. **Any student who does not present will receive a Presentation Score of 0.**



Q & A

